

PENSAMIENTO COMPUTACIONAL

Juan Esteban Campos Cardona

1091203999

Nicolas Santiago Guzman Chala

1126907455

Yulian Stiven Diaz Pulido

1091203910



Profesora:

Carolina Londoño Idarraga

Universidad del Quindío

Facultad de ingeniería

Ingeniería de sistemas y computación

Programación 2

Armenia, Mayo de 2026

1. ¿Qué se solicita finalmente?

Se solicita desarrollar una plataforma de gestión de eventos y venta de entradas en Java, que permita a usuarios y administradores interactuar con el sistema según sus permisos.

El sistema debe permitir:

- explorar eventos,
- gestionar recintos, zonas y asientos,
- realizar compras y pagos,
- agregar servicios adicionales,
- administrar incidencias,
- generar reportes y métricas,
- y aplicar patrones de diseño junto con principios SOLID.

Además, la solución debe implementar una arquitectura modular y escalable utilizando patrones creacionales, estructurales y de comportamiento.

2. ¿Qué información es relevante y como se agrupa?

Usuario

La entidad Usuario almacena información relevante como idUsuario, nombre completo, correo electrónico, número de teléfono, métodos de pago simulados e historial de compras.

Eventos

La entidad Evento almacena información relevante como idEvento, nombre del evento, categoría, descripción, ciudad, fecha y hora, estado del evento, políticas de cancelación y reembolso, además del recinto asociado donde se realizará.

Recinto

La entidad Recinto representa el lugar físico donde se realiza un evento. Almacena información como idRecinto, nombre del recinto, dirección, ciudad y Capacidad

Zona

La entidad Zona representa un sector del recinto Incluye identificador único (idZona), nombre, capacidad, precio base y configuración de asientos (si aplica).

Asientos

La entidad Asiento representa una unidad individual dentro de una zona. Almacena información como idAsiento, fila, número y estado del asiento.

Compras

La entidad Compra representa la adquisición de entradas realizada por un usuario. Almacena información como idCompra, usuario asociado, evento asociado, fecha de creación, total de la compra, estado y servicios adicionales seleccionados.

Entradas

La entidad Entrada representa cada ticket generado después de una compra. Almacena información como idEntrada, zona, asiento asociado, precio final y estado de la entrada.

Pago

La entidad Pago almacena información relacionada con las transacciones realizadas por los usuarios, incluyendo método de pago, estado del pago, comprobantes y valor pagado.

Tarifa

La entidad Tarifa administra la información relacionada con precios base, costos adicionales y modificaciones dinámicas del valor de las entradas.

Incidencia

La entidad Incidencia almacena información relacionada con problemas o situaciones excepcionales ocurridas dentro de la plataforma. Incluye datos como idIncidencia, tipo, descripción, fecha y entidad afectada.

4. ¿Qué funcionalidades se solicitan?

El usuario puede registrarse e iniciar sesión en la plataforma, modificar la información de su perfil, consultar eventos disponibles, visualizar detalles de eventos, seleccionar entradas, realizar compras, gestionar pagos y consultar el historial de sus compras.

El evento permite gestionar la información relacionada con conciertos, obras de teatro o conferencias disponibles en la plataforma. También permite consultar disponibilidad de zonas y asientos, controlar cambios de estado como publicación, pausa o cancelación, y proporcionar información detallada a los usuarios sobre precios, reglas y capacidad del recinto.

El recinto permite organizar la distribución física del evento, administrar las zonas asociadas y controlar capacidades generales y disponibilidad de espacios.

La zona permite organizar y determinar el precio de los asientos y controlar diferentes tipos de asientos

El asiento permite controlar disponibilidad y estados como disponible, reservado, vendido o bloqueado, evitando conflictos durante la compra de entradas.

La compra permite gestionar el proceso de selección de entradas, modificación de pedidos antes del pago, cancelaciones, cálculo de costos y seguimiento del estado de la transacción.

La entrada permite validar el acceso a eventos, consultar información relacionada con la compra y controlar estados como activa, usada o anulada.

El pago permite procesar transacciones utilizando diferentes estrategias de pago y validar el resultado de cada operación realizada dentro de la plataforma.

La tarifa permite calcular el costo final de una compra teniendo en cuenta servicios adicionales como acceso VIP, seguros, merchandising o parqueadero.

La incidencia permite registrar errores de pago, intentos de doble compra, cancelaciones masivas u otros eventos operativos importantes, facilitando el seguimiento y control administrativo del sistema.

1. ¿Qué debo hacer para probar las funcionalidades?

Se deben crear pruebas funcionales y casos de uso simulados para validar el comportamiento del sistema.

Casos de prueba principales

Gestión de usuarios

- registrar usuario
- iniciar sesión
- actualizar perfil

Gestión de eventos

- crear evento

- publicar evento
- cancelar evento

Gestión de asientos

- reservar asiento disponible
- impedir compra de asiento vendido
- bloquear asientos

Gestión de compras

- crear compra
- modificar compra antes del pago
- cancelar compra

Pagos

- procesar pagos con distintos métodos
- validar comprobantes

Servicios adicionales

- agregar VIP
- agregar seguro
- recalcular total

Notificaciones

- cambios de estado del evento
- cambios de estado de la compra

Reportes

- generar CSV
- generar PDF

¿Qué puedo reutilizar?

En el desarrollo de la plataforma se han implementado distintos patrones de diseño con el objetivo de facilitar la organización del sistema, mejorar su mantenibilidad y permitir la reutilización de componentes.

En primer lugar, se utilizó el patrón Singleton, el cual permite asegurar una única instancia encargada de gestionar la información global de la aplicación, centralizando el acceso a los datos.

Para la creación de eventos, se aplicó el patrón Abstract Factory, el cual facilita la generación de distintos tipos de eventos de manera estructurada y consistente. Complementariamente, el patrón Builder se emplea para simplificar la construcción de eventos, permitiendo manejar de forma flexible aquellos atributos opcionales sin necesidad de sobrecargar el proceso de creación.

En la estructura de zonas y asientos se implementó el patrón Composite, ya que una zona está compuesta por múltiples sillas, lo que permite tratarlas de manera uniforme dentro del sistema.

El patrón Decorator se utilizó en la gestión de compras, permitiendo agregar servicios adicionales como acceso VIP, parqueadero o merchandising de forma dinámica sin modificar la estructura base de la compra.

Para la generación de reportes, se implementó el patrón Adapter, el cual permite integrar una API externa de generación de informes de manera compatible con la estructura del sistema, evitando conflictos entre ambas arquitecturas.

El manejo de métodos de pago en las compras se resolvió mediante el patrón Strategy, permitiendo aplicar diferentes formas de pago según el contexto del usuario sin alterar la lógica principal de la compra.

Por otro lado, el patrón Observer se utilizó para la gestión de incidencias, permitiendo notificar automáticamente a diferentes partes del sistema cuando ocurre un evento relevante, como una cancelación o un error en una operación.

Finalmente, el patrón State se aplicó en la gestión de asientos, permitiendo controlar su comportamiento según su estado (disponible, reservado, vendido o bloqueado), facilitando así la validación de acciones sobre ellos.

Enlace al diagrama de clases:

https://miro.com/app/board/uXjVHV98fg0=/?share_link_id=839295507157